



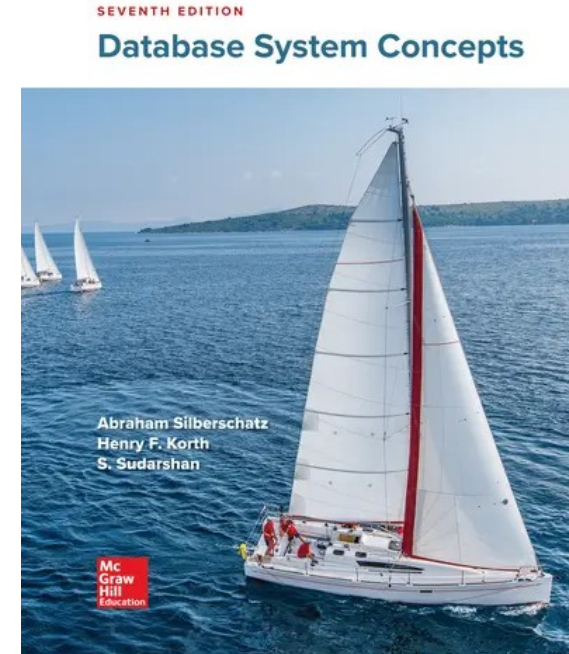
UNIVERSITY
OF TRENTO

Introduction to database concepts

Databases course

Notice

- This slide set is a modified version of the one provided with the book Database System Concepts (7th edition) by A. Silberschatz, H. Korth, and S. Sudarshan, published by McGraw-Hill, ISBN: 978-0078022159, 2020
- This slideset covers
 - chapter 1 of the *Database System Concepts* book



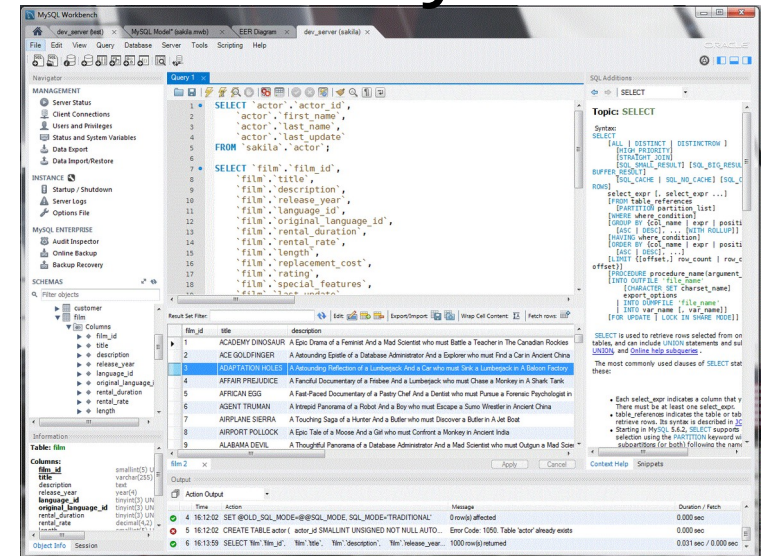
Databases



A structured collection of data stored in a computer system, managed by specialized software (DBMS) for efficient, reliable, and multi-user access

DBMS: Database Management System

How do they look like



Database Applications Examples

- Enterprise Information
 - Sales: customers, products, purchases
 - Accounting: payments, receipts, assets
 - Human Resources: Information about employees, salaries, payroll taxes
- Manufacturing: management of production, inventory, orders, supply chain
- Banking and finance
 - Customer information, accounts, loans, and banking transactions.
 - Credit card transactions
 - Finance: sales and purchases of financial instruments (e.g., stocks and bonds; storing real-time market data)
- Universities: registration, grades

Database Applications Examples (cont.)

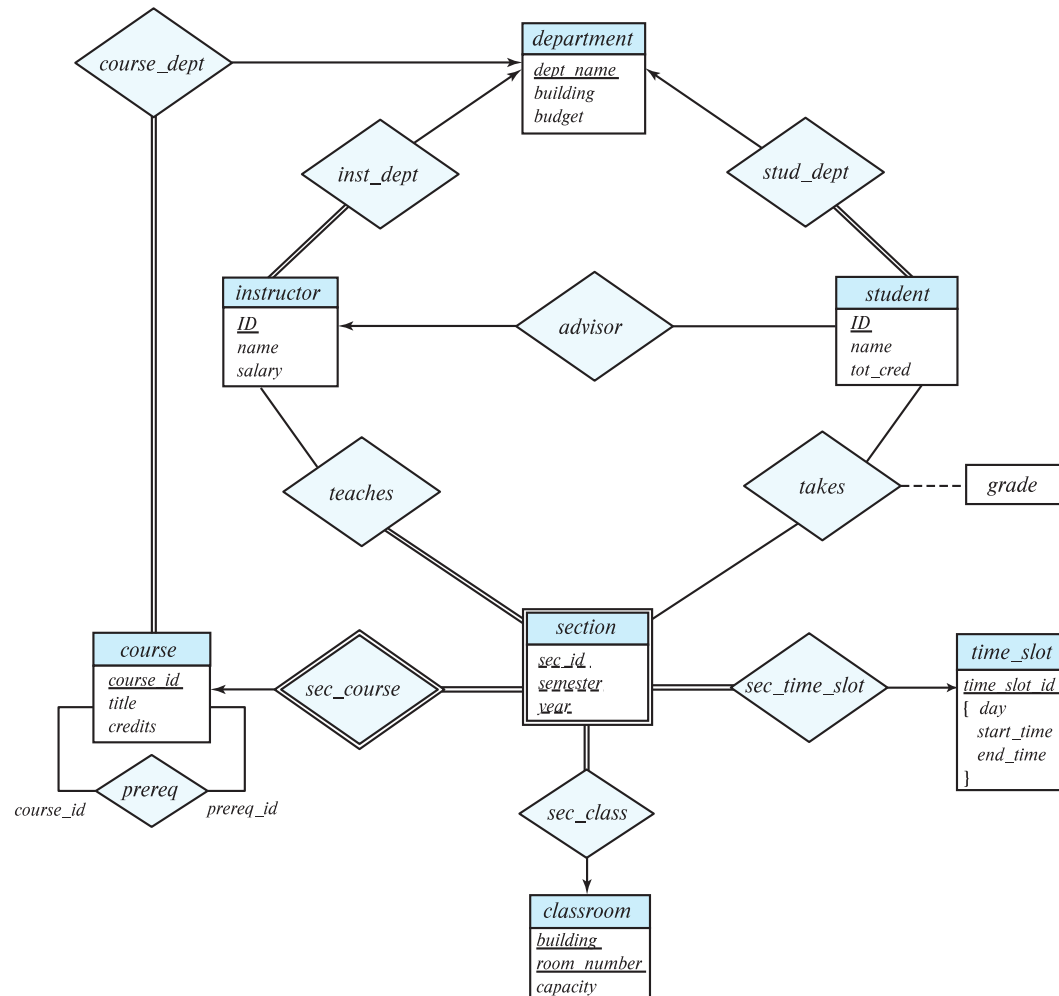
- Airlines: reservations, schedules
- Telecommunication: records of calls, texts, and data usage, generating monthly bills, maintaining balances on prepaid calling cards
- Web-based services
 - Online retailers: order tracking, customized recommendations
 - Online advertisements
- Document databases
- Navigation systems: For maintaining the locations of various places of interest along with the exact routes of roads, train systems, buses, etc.

Models and Design

View of Data

- A database system is a **collection of interrelated data and a set of programs that allow users to access and modify these data**
- Major purpose: **provide users with an abstract view of the data**
 - The system hides certain details of *how* data are stored and maintained
- Two main instruments
 - **Data models**
 - A collection of conceptual tools for describing data, data relationships, data semantics, and consistency constraints
 - **Data abstraction**
 - Methods to hide the complexity of data structures representing data in the database through several levels of data abstraction
 - Needed to retrieve data efficiently and smoothly

Data Models: ER model



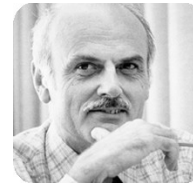
- *Entity-Relationship (ER)*
 - Conceptual design of database
 - Collection of basic objects, called **entities**, and **relationships among them**

Data Models: Relational Model

- *Relational*: data implementation
 - Uses a collection of **tables** (**relations**) to represent both data and the relationships among data
 - Example of record-based model: fixed-format records of several types
 - Most widely used data model for database systems

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table



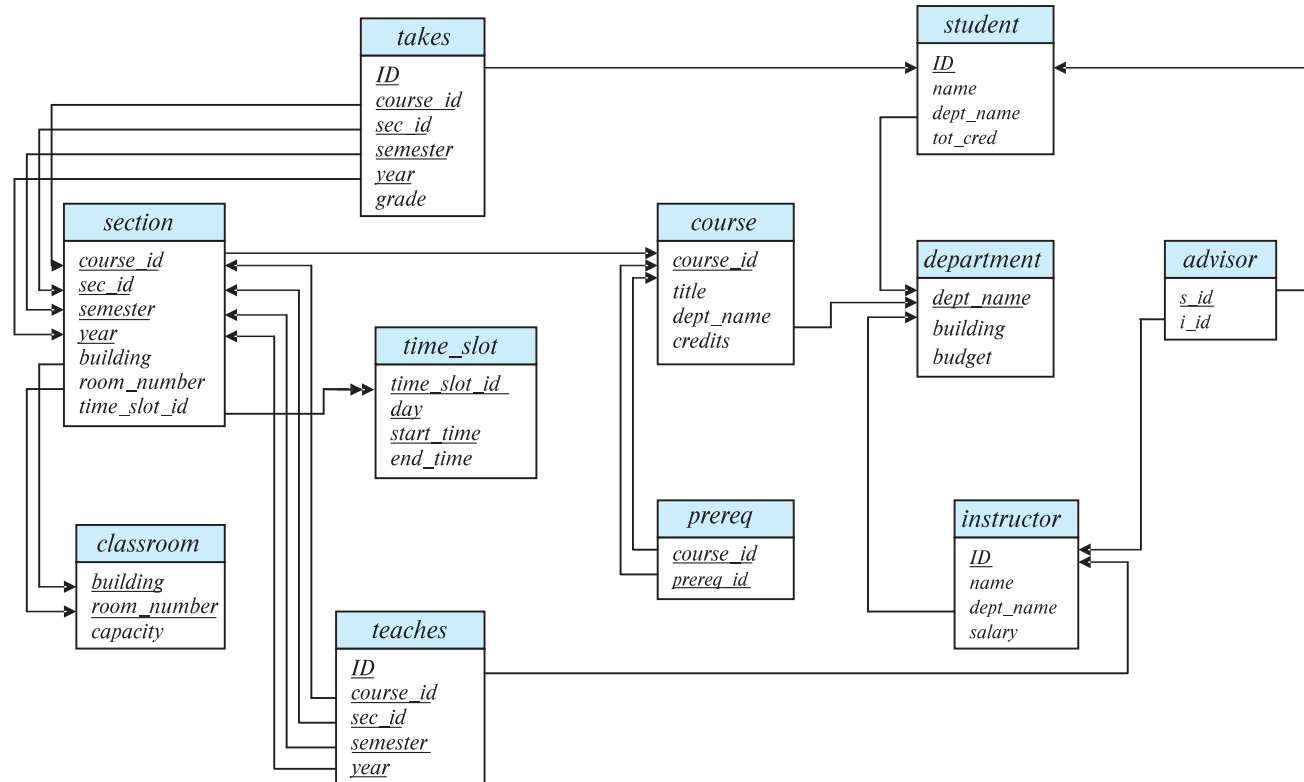
Ted Codd
Turing Award
1981

Data Models: Other Models

- *Object-based* data models (Object-oriented and Object-relational)
 - Merges programming language objects with databases
- *Semi-structured* data model (JSON or XML)
 - Flexible schemas
- Other older models (pre-relational DBMS)
 - Network model: data represented as graphs
 - Hierarchical model: data represented as trees

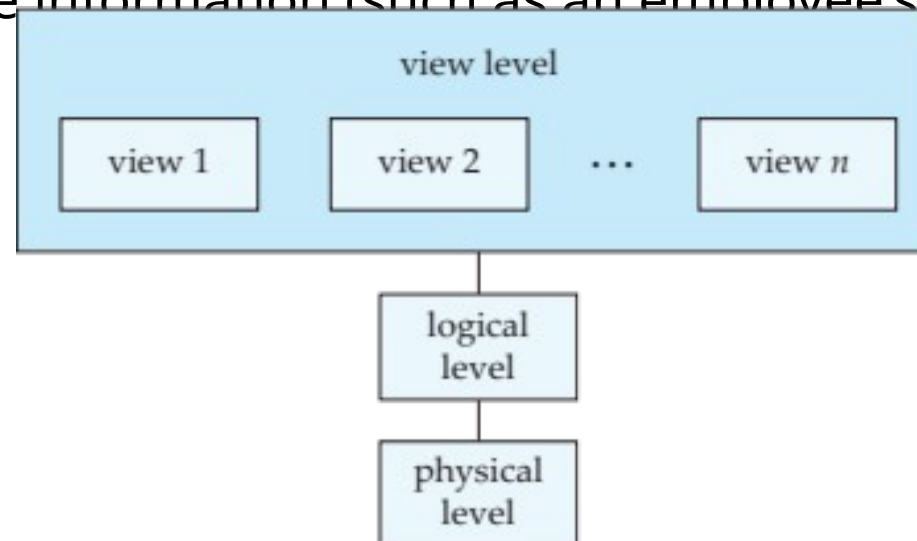
University Database Example

- The reference text uses university database to illustrate concepts
- Data consists of information about students, instructors, classes
- Application program examples
 - Add new students, instructors, and courses
 - Register students for courses, and generate class rosters
 - Assign grades to students, compute grade point averages (GPA) and generate transcripts



Levels of Abstraction

- **Physical level:** describes *how a record is stored* (e.g., instructor)
- **Logical level:** describes *what data is stored* in database, and the relationships among such data
 - Describes the entire database in terms of a small number of simple structures, independently from the physical level
- **View level:** *what users see*, application programs hide details of data types
 - Views can also hide information (such as an employee's salary) for security purposes



Levels of Abstraction - an analogy

- Analogy with data types in programming languages: with structured types we could describe a record as

```
type instructor = record
```

```
  ID : string;
```

```
  name : string;
```

```
  dept_name : string;
```

```
  salary : integer;
```

```
end;
```

- University might have several such records, e.g.
 - *department* – fields: *dept_name*, *building*, *budget*
 - *course* – fields *course_id*, *title*, *dept_name*, *credits*
 - *student* – fields *ID*, *name*, *dept_name*, *tot_cred*

Levels of Abstraction - an analogy (Cont.)

- **Physical level**

- Records (e.g. instructor): blocks of consecutive bytes, compiler hides this from user
- In databases (DBs), a table can be a sequence of records in a file with variable size, retrieved efficiently through data structures such as indexes, all hidden from DB's user

- **Logical level**

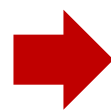
- Every record is described with its type and interrelation among records are expressed through programming language
- In databases, types are decided in the design of the DB, and relationship across records is defined at the logical level

- **View level**

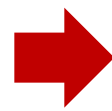
- Computer users see set of application programs that hide details of files, structures, data, and more
- In databases we create views that provide access to certain data types and hide details of logical level

Database Data-Driven Design

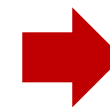
Application
Requirements



Conceptual
Design



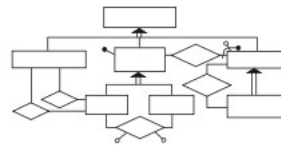
Logical
Design



Physical
Design



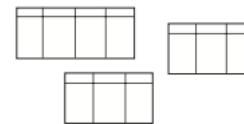
Conceptual
Schema



ER model to capture entities, relationships, attributes, and constraints (independent of implementation)



Logical
Schema



Translate that ER design into a **specific data model** (usually relational): tables, attributes, keys, foreign keys



Physical
Schema



DBA (or DBMS) **decides how those tables are stored**: file structures, indexes, clustering, partitioning, etc

Instances and Schemas

- **Physical schema**

- Different structures affect performance
 - **Heap file** → must scan many rows
 - **B-tree index** → fast lookups and range queries
 - **Hash index** → very fast lookups on exact values
- Physical data **independence** – the ability to modify the physical schema without changing the logical schema

- **Logical Schema**

- Example: The database consists of information about a set of customers and accounts in a bank and the relationship between them
 - Analogous to type information of a variable in a program
- **Instance** – the **actual content of the database at a particular point in time**
 - Analogous to the value of a variable

Languages

Database languages

- **Data-definition language (DDL)**
 - Specify database schema
- **Data-manipulation language (DML)**
 - Express database queries and updates
- In practice, not two separate languages: **Structured Query Language (SQL)** covers both

Data Definition Language (DDL)

- Specification notation for defining the **database schema**
- Data stored must satisfy certain constraints that can be defined with DDL and checked every time the database is updated
 - **Domain constraints:** declare an attribute to be of a specific domain act as a constraint on the values it can take (integrity constraints)
 - **Referential integrity:** a value that appears in one relation for a given set of attribute also appears in a certain set of attributes of another relation
 - **Authorization:** read, insert, update, delete authorizations for different users
- Example of SQL DDL

```
create table instructor (  
    ID                char(5),  
    name              varchar(20),  
    dept_name         varchar(20),  
    salary           numeric(8,2))
```

Data Manipulation Language (DML)

- Language for accessing and updating the data organized by the appropriate data model
 - DML also known as **query language**
- There are basically two types of data-manipulation language
 - **Procedural DML**: specify **what** data are needed **and how** to get those data, e.g.
for each instructor in instructor_table:
if instructor.dept_name = 'Physics':
print instructor.name
 - **Declarative DML (non-procedural)**: specify **what** data are needed **without** specifying **how** to get those data, e.g.
select name
from instructor
where dept_name = 'Physics';

SQL Query Language



- SQL query language is declarative (non-procedural)
 - A query takes as input several tables (possibly only one) and always returns a single table
- Example to find all instructors in Comp. Sci. dept

```
select name
from instructor
where dept_name = 'Comp. Sci.'
```
- To be able to compute complex functions SQL can be embedded in some higher-level language
- Application programs generally access databases through one of
 - Language extensions to allow embedded SQL
 - Application program interface (e.g., ODBC/JDBC) which allow SQL queries to be sent to a database

Where NoSQL Fits



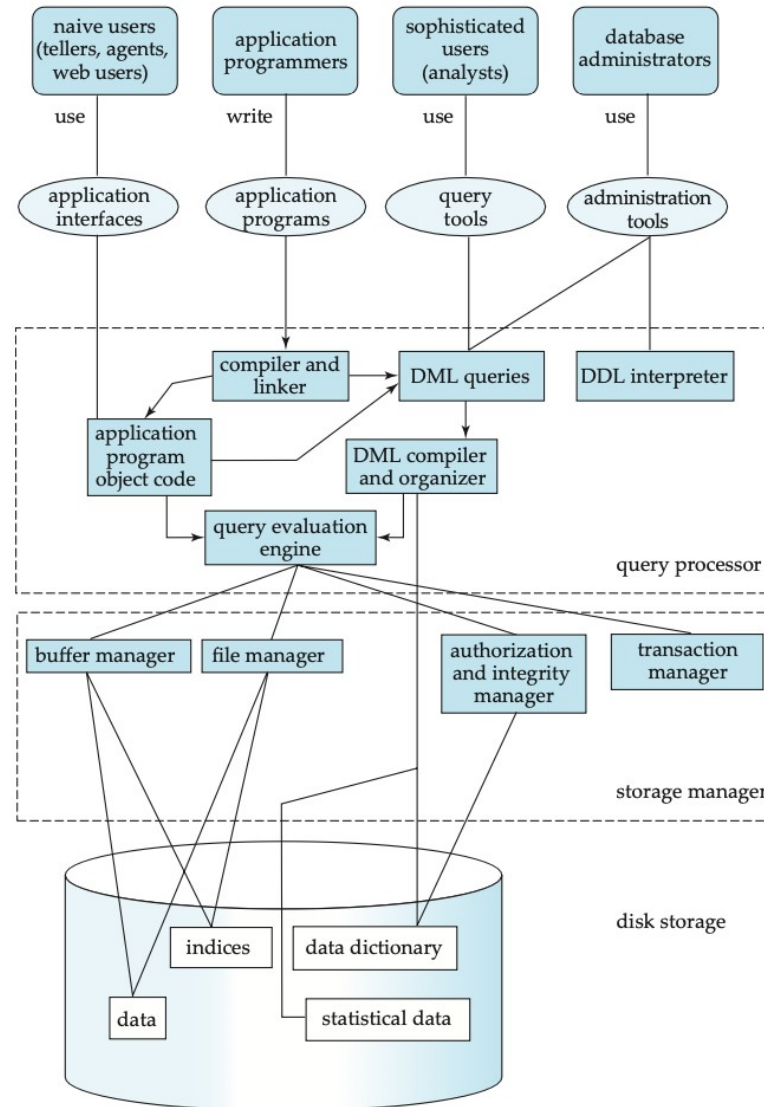
- Emerged in 2000s (web-scale apps, big data)
 - Relational DBs still dominate, but NoSQL fills new needs
- Not one single model → families of data models:
 - **Document** (JSON → MongoDB)
 - **Key-value** (Redis, DynamoDB)
 - **Column-family** (Cassandra, HBase)
 - **Graph** (Neo4j)
- Often schema-flexible, distributed, high-performance

Database Systems

Database System

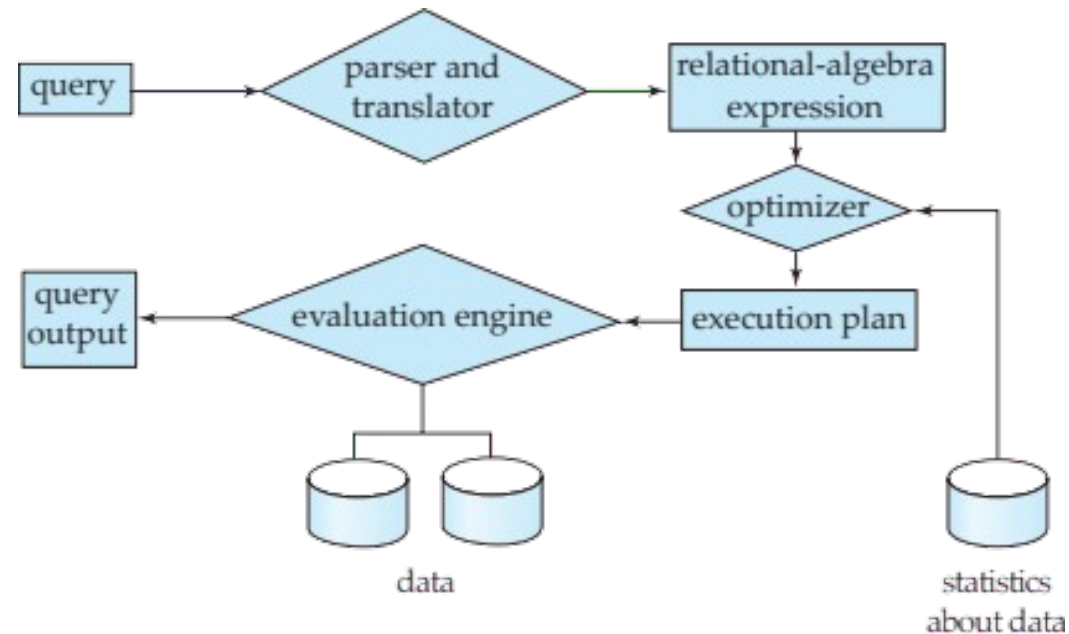
- A database system is partitioned into modules that deal with each of the responsibilities of the overall system
- The functional components of a database system can be divided into two main components
 - Query processor
 - Storage manager

Database System Representation



Functions by Query Processor

1. **Parsing** (check syntax, verifies relation names are OK, etc.) and translation into relation-algebra
2. **Optimization** picks the lowest cost evaluation plan from among the various alternatives
3. **Evaluation** executes low-level instructions generated by the DML compiler



Storage Manager

- Interface between the low-level data stored in the database and the application programs and queries submitted to the system
- Storage manager is responsible to the following tasks
 - Interaction with the OS file manager
 - Efficient storing, retrieving and updating of data
- Storage manager components include:
 - **Authorization and integrity manager** (read, insert, delete, update by user or role)
 - **Transaction manager**
 - **File manager**
 - **Buffer manager**

Storage Manager: Data Structures

- The storage manager implements several data structures as part of the physical system implementation:
 - Data files: store the database itself
 - Data dictionary: stores metadata about the structure of the database, in particular the schema of the database
 - Indices: can provide fast access to data items
 - A database index provides pointers to those data items that hold a particular value

Storage Manager: Transaction Management

- A **transaction** is a collection of operations that performs a single logical function in a database application
 - Each transaction is a unit of both atomicity and consistency
- **Transaction manager** component ensures that the database remains in a consistent (correct) state despite system failures (e.g., power failures and operating system crashes) and transaction failures
 - **Concurrency-control manager** controls the interaction among the concurrent transactions, to ensure the consistency of the database
 - **Failure recovery manager** detects system failures and restores the database to the state that existed prior to the occurrence of the failure

Database Architecture (not covered)

- Centralized databases
 - One to a few cores, shared memory
- Client-server
 - One server machine executes work on behalf of multiple client machines
- Parallel databases
 - Many core shared memory
 - Shared disk
 - Shared nothing
- Distributed databases
 - Geographical distribution
 - Schema/data heterogeneity

Bits of History

History of Database Systems

- 1950s and early 1960s
 - Data processing using magnetic tapes for storage
 - Tapes provided only sequential access
 - Punched cards for input
- Late 1960s and 1970s
 - Hard disks allowed direct access to data
 - Network and hierarchical data models in widespread use
 - Ted Codd defines the relational data model
 - Would win the ACM Turing Award for this work
 - IBM Research begins System R prototype
 - UC Berkeley (Michael Stonebraker) begins Ingres prototype
 - Oracle releases first commercial relational database
 - High-performance (for the era) transaction processing

History of Database Systems (Cont.)

- 1980s
 - Research relational prototypes evolve into commercial systems
 - SQL becomes industrial standard
 - Parallel and distributed database systems
 - Wisconsin, IBM, Teradata
 - Object-oriented database systems
- 1990s
 - Large decision support and data-mining applications
 - Large multi-terabyte data warehouses
 - Emergence of Web commerce

History of Database Systems (Cont.)

- 2000s
 - Big data storage systems
 - Google BigTable, Yahoo PNuts, Amazon,
 - “NoSQL” systems.
 - Big data analysis: beyond SQL
 - Map reduce and friends
- 2010s
 - SQL reloaded
 - SQL front end to Map Reduce systems
 - Massively parallel database systems
 - Multi-core main-memory databases